

SHUNYA Frontend — Complete Duplication Map

Generated: LX-06 Architectural Convergence Audit
Files scanned: 194 in `/home/shunya-deploy/shunya_os/frontend/src/`

1. COMPLETE FILE TAXONOMY BY CATEGORY

api/ — 15 files

File	Purpose	Status
<code>api/ai-chat.ts</code>	AI Chat service — POST <code>/api/v1/ai/chat</code>	Canonical
<code>api/client.ts</code>	Generic API client (signin, auth pages, generic query)	DUPLICATE — overlaps with <code>fetch-with-auth</code>
<code>api/event-bus.ts</code>	Frontend event bus (Map-based pub/sub)	DUPLICATE — two event bus implementations exist
<code>api/fetch-with-auth.ts</code>	Auth-wrapped fetch helper (X-Identity-Id header)	DUPLICATE — overlaps with <code>client.ts</code> auth pattern
<code>api/integrations.ts</code>	Integrations API client (social, ads, content, proxy)	Canonical
<code>api/objects.ts</code>	Object CRUD API client (workspaces, objects, upload)	Canonical
<code>api/profile.ts</code>	Multi-profile switching manager	Canonical
<code>api/session.ts</code>	Session manager (in-memory + sessionStorage)	DUPLICATE — overlaps with <code>supabase-session</code>
<code>api/supabase-session.ts</code>	Supabase session bridge	DUPLICATE — overlaps with <code>session.ts</code>
<code>api/supabase.ts</code>	Supabase client & auth helpers	Canonical
<code>api/use-ai-presence.ts</code>	<code>useAIPresence</code> hook — polls AI insights every 45s	Canonical
<code>api/use-query.ts</code>	<code>useQuery</code> + <code>useMutation</code> hooks (generic data fetching)	Canonical
<code>api/use-realtime-sync.ts</code>	<code>useRealtimeSync</code> — polls <code>/api/v1/events</code> every 15s	Canonical
<code>api/use-workspace-memory.ts</code>	Workspace UI state persistence (sessionStorage)	Canonical
<code>api/workspace.ts</code>	Workspace CRUD API + active workspace tracking	Canonical

context/ — 1 file

File	Purpose	Status
<code>context/WorkspaceContext.tsx</code>	React Context for workspace state + <code>WorkspaceProvider</code>	LEGACY — being replaced by runtime store

hooks/ — 2 files

File	Purpose	Status
<code>hooks/runtime-hooks.ts</code>	React bridge to orchestrator (<code>useRuntime</code> , <code>useRuntimes</code> , etc.)	Canonical
<code>hooks/workspace-hooks.ts</code>	React bridge to workspace Zustand store	Canonical

lib/ — 2 files

File	Purpose	Status
<code>lib/component-registry.ts</code>	Dynamic component registry (Map-based)	Canonical
<code>lib/event-bus.ts</code>	Class-based <code>EventBus</code> (<code>onAny</code> , <code>listenerCount</code>)	DUPLICATE — second event bus implementation

runtimes/ — 27 files

File	Purpose	Status
<code>runtimes/event-bus.ts</code>	Typed runtime event bus (<code>RuntimeEvent</code> union type)	DUPLICATE — THIRD event bus
<code>runtimes/orchestrator.ts</code>	Runtime orchestrator — lifecycle, DAG, health	Canonical
<code>runtimes/registration.ts</code>	Registers all runtimes with orchestrator	Canonical
<code>runtimes/state-fabric.ts</code>	State fabric — versioned state, snapshots, persistence	Canonical
<code>runtimes/module-registry.ts</code>	Module registry — dynamic module loading	Canonical
<code>runtimes/workspace/store.ts</code>	Zustand workspace store (runtime version)	Canonical

<code>runtimes/workspace/types.ts</code>	Workspace types + state machine	Canonical
<code>runtimes/commitment/engine.ts</code>	Commitment runtime engine	Canonical
<code>runtimes/composition/engine.ts</code>	Composition engine	Canonical
<code>runtimes/conversation/engine.ts</code>	Conversation runtime engine	Canonical
<code>runtimes/experience/engine.ts</code>	Experience engine	Canonical
<code>runtimes/graph/engine.ts</code>	Object graph runtime	Canonical
<code>runtimes/intelligence/engine.ts</code>	Intelligence runtime	Canonical
<code>runtimes/layout/engine.ts</code>	Layout engine	Canonical
<code>runtimes/object/engine.ts</code>	Object runtime	Canonical
<code>runtimes/timeline/engine.ts</code>	Timeline runtime	Canonical
<code>runtimes/modules/business.ts</code>	Business module (implements ShunyaModule)	Canonical
<code>runtimes/modules/ubme/</code>	UBME module (10 files)	Canonical

tokens/ — 2 files

File	Purpose	Status
<code>tokens/definitions.ts</code>	Design token definitions (source of truth)	Canonical
<code>tokens/token-provider.tsx</code>	React TokenProvider component	Canonical

types/ — 1 file

File	Purpose	Status
<code>types/index.ts</code>	Core TypeScript types	Canonical

Root files — 3 files

File	Purpose	Status
<code>app.tsx</code>	App entry point + routing + MantineProvider	Canonical
<code>main.tsx</code>	React root mount + ErrorBoundary + SW	Canonical
<code>vite-env.d.ts</code>	Vite type declarations	Canonical

2. DUPLICATE PAIRS — COMPLETE ANALYSIS

DUPLICATE SET 1: THREE EVENT BUS IMPLEMENTATIONS

Files:

- `api/event-bus.ts` — **Frontend Event Bus** (Map>, auto-emits 'notification' on every event)
- `lib/event-bus.ts` — **Class-based EventBus** (onAny, listenerCount, wildcard handlers)
- `runtimes/event-bus.ts` — **Runtime Event Bus** (typed RuntimeEvent union, onAny, clear)

Import Chains:

- `api/event-bus.ts` ← imported by: `api/use-ai-presence.ts`, `api/use-query.ts`, `api/use-realtime-sync.ts`
- `lib/event-bus.ts` ← imported by: (none found — likely dead)
- `runtimes/event-bus.ts` ← imported by: `runtimes/orchestrator.ts`, `runtimes/state-fabric.ts`, `runtimes/registration.ts`, `runtimes/workspace/store.ts`, `components/workspace/workspace-container.tsx`

Verdict:

Status	File	Reason
CANONICAL	<code>runtimes/event-bus.ts</code>	Typed events, used by all runtimes, integrated with orchestrator
DUPLICATE	<code>api/event-bus.ts</code>	Inline stylesheet injection, auto-notification side effects, used by old API hooks
DEAD	<code>lib/event-bus.ts</code>	No imports found, no runtime integration

Action: Merge `api/event-bus.ts` consumers onto `runtimes/event-bus.ts` ; remove `lib/event-bus.ts` .

DUPLICATE SET 2: TWO COMMAND SURFACES

Files:

- 1. `components/executive-home/command-surface.tsx` — **Command Surface (Executive Home)** — `/open` , `/create` , `/new` , voice recognition, AI intelligence query
- 2. `components/living-workspace/command-surface.tsx` — **Living Command Surface (LX-01)** — contextual suggestions, framer-motion, action execution, living store integration
- 3. `components/ai/command-bar.tsx` — **AI Command Bar 3.0** — Multi-step workflows, undo stack, context memory, scheduling, global search, full CRUD
- 4. `components/ai/command-palette.tsx` — **Command Palette (Mantine Spotlight)** — `Cmd+K`, navigation actions, Mantine Spotlight

Duplicate Analysis:

Status	File	Reason
CANONICAL	<code>components/living-workspace/command-surface.tsx</code>	LX-01 canonical, uses living store, framer-motion
LEGACY	<code>components/executive-home/command-surface.tsx</code>	E1 milestone, being replaced by LX-01
DUPLICATE	<code>components/ai/command-bar.tsx</code>	Overlaps with living-workspace command surface (same CRUD, same AI chat)
COMPLEMENTARY	<code>components/ai/command-palette.tsx</code>	Mantine Spotlight — different UX pattern (palette vs. bar), could coexist

Action: Remove `components/ai/command-bar.tsx` (functionality absorbed by living command surface). Keep command-palette as `Cmd+K` shortcut layer. Remove `components/executive-home/command-surface.tsx` after migration.

DUPLICATE SET 3: TWO AI COPILOT / AI PRESENCE PANELS

Files:

- 1. `components/copilot/ai-copilot.tsx` — **AI Copilot** — context-aware assistant, proactive insights via `ModuleRegistry`
- 2. `components/living-workspace/ai-presence-panel.tsx` — **LX-02A Continuous Companion** — observations, recommendations, execution tracking
- 3. `components/ai/ai-insights.tsx` — **AI Business Insights** — Mantine Edition, fetches all objects, sends to AI chat, displays insights

Verdict:

Status	File	Reason
CANONICAL	<code>components/living-workspace/ai-presence-panel.tsx</code>	LX-02A canonical, living store integration, complete companion
DUPLICATE	<code>components/copilot/ai-copilot.tsx</code>	Same concept (context-aware AI), less evolved, no living-store
COMPLEMENTARY	<code>components/ai/ai-insights.tsx</code>	Focused on business metrics analysis — different scope

Action: Remove `components/copilot/ai-copilot.tsx` (absorbed by ai-presence-panel).

DUPLICATE SET 4: TWO LIVING WORKSPACE COMPONENTS

Files:

- 1. `components/living-workspace/living-workspace.tsx` — **LX-01 Canonical Living Workspace** — top bar, executive briefing, AI presence, reality stream, living objects, command surface
- 2. `components/workspace/living-workspace.tsx` — **SX-11 LivingWorkspace** — object-specific cognitive workspace with insights, recommendations, timeline

Verdict:

These are actually different concepts: - `components/living-workspace/` = The **new OS experience** (the "living workspace" as the entire operating system) - `components/workspace/living-workspace.tsx` = An **object detail workspace** within the old OS

Status	File	Reason
CANONICAL	<code>components/living-workspace/</code>	The new LX-01 canonical OS
LEGACY	<code>components/workspace/living-workspace.tsx</code>	Old workspace system, will be replaced

DUPLICATE SET 5: TWO WORKSPACE BAR / OS BAR COMPONENTS

Files:

- 1. `components/public/os-bar.tsx` — **OS Bar (DaisyUI)** — mode tabs, auth controls, theme toggle

2. `components/workspace/workspace-bar.tsx` — **OS Command Bar** — AI presence, search, workspace switcher, notifications, profile

Verdict:

Status	File	Reason
CANONICAL	<code>components/workspace/workspace-bar.tsx</code>	Runtime-integrated, workspaces, AI, notifications
LEGACY	<code>components/public/os-bar.tsx</code>	Old DaisyUI version, used by legacy homepage

DUPLICATE SET 6: THREE SESSION/AUTH MECHANISMS

Files:

- `api/session.ts` — **SessionManager** (in-memory + sessionStorage)
- `api/supabase-session.ts` — **SupabaseSessionBridge** (Supabase session → sessionStorage)
- `api/supabase.ts` — **Supabase Client** (direct Supabase auth with `signInWithEmail`, `signInWithGoogle`, etc.)

Verdict:

Status	File	Reason
CANONICAL	<code>api/supabase.ts</code>	The real auth provider
DUPLICATE	<code>api/supabase-session.ts</code>	Bridges Supabase to old session pattern — should be eliminated
LEGACY	<code>api/session.ts</code>	Old custom session pattern (superseded by Supabase)

DUPLICATE SET 7: TWO AUTH-FETCH MECHANISMS

Files:

- `api/client.ts` — **Generic API Client** — signin, auth pages, generic query
- `api/fetch-with-auth.ts` — **fetchWithAuth** — wraps fetch with X-Identity-Id

Verdict:

Status	File	Reason
CANONICAL	<code>api/client.ts</code>	More structured, typed endpoints
DUPLICATE	<code>api/fetch-with-auth.ts</code>	Same auth header pattern, used by random panels
DUPLICATE	<code>api/objects.ts</code> (internal auth)	Has its OWN auth headers pattern built in

Action: Standardize on `api/client.ts` for all API calls; remove `api/fetch-with-auth.ts`.

DUPLICATE SET 8: TWO CONTEXT PANELS / AI RESIDENT

Files:

- `components/workspace/context-panel.tsx` — **Context Panel** with AI Resident section, ModuleRegistry integration
- `components/workspace/workspace-shell.tsx` — **Workspace Shell** (three-zone layout wrapping context panel)

No duplication here — these are different concerns (shell vs panel content).

DUPLICATE SET 9: TWO REGISTRY SYSTEMS

Files:

- `components/registry.ts` — **Component Registry** — 12 registered components (Metric, Badge, StatusDot, etc.)
- `lib/component-registry.ts` — **Component Registry (class-based)** — Map registry, by category, metadata
- `components/space-registry.ts` — **Space Registry** — lazy-loaded space panels

Verdict:

Status	File	Reason
DUPLICATE	<code>components/registry.ts</code>	Static array, same concept as lib/component-registry
CANONICAL	<code>lib/component-registry.ts</code>	Class-based, more flexible, supports metadata
COMPLEMENTARY	<code>components/space-registry.ts</code>	Different purpose (lazy loading spaces vs component discovery)

Action: Merge `components/registry.ts` into `lib/component-registry.ts` .

3. IMPORT DEPENDENCY CHAINS

api/ dependency graph:

```
api/supabase.ts ← api/supabase-session.ts
api/session.ts ← api/workspace.ts ← context/WorkspaceContext.tsx
api/event-bus.ts ← api/use-ai-presence.ts
api/event-bus.ts ← api/use-query.ts
api/event-bus.ts ← api/use-realtime-sync.ts
api/ai-chat.ts ← components/ai/ai-insights.tsx
api/ai-chat.ts ← components/ai/command-bar.tsx
api/ai-chat.ts ← components/ai/file-assistant.tsx
api/objects.ts ← components/ai/command-bar.tsx
api/workspace.ts ← context/WorkspaceContext.tsx
api/client.ts ← app.tsx ← main.tsx
```

runtimes/ dependency graph:

```
runtimes/event-bus.ts ← runtimes/orchestrator.ts
runtimes/event-bus.ts ← runtimes/state-fabric.ts
runtimes/event-bus.ts ← runtimes/workspace/store.ts
runtimes/event-bus.ts ← components/workspace/workspace-container.tsx
runtimes/orchestrator.ts ← runtimes/registration.ts
runtimes/orchestrator.ts ← hooks/runtime-hooks.ts
runtimes/workspace/store.ts ← hooks/workspace-hooks.ts
runtimes/workspace/types.ts ← runtimes/workspace/store.ts
```

Component dependency (key chains):

```
app.tsx → context/WorkspaceContext.tsx
app.tsx → components/living-workspace (LX-01)
app.tsx → components/public/homepage.tsx (legacy)

components/public/homepage.tsx → api/client.ts, api/session.ts, api/event-bus.ts
components/public/homepage.tsx → components/ai/command-bar.tsx
components/public/homepage.tsx → components/notifications/notification-bell.tsx
components/public/homepage.tsx → api/use-ai-presence.ts, api/use-realtime-sync.ts
components/public/homepage.tsx → components/space-registry.ts (lazy-loaded panels)

components/workspace/workspace-container.tsx → runtimes/event-bus.ts
components/workspace/workspace-container.tsx → runtimes/module-registry.ts
components/workspace/workspace-container.tsx → hooks/workspace-hooks.ts
```

4. ALL ZUSTAND STORES

Store	File	State Managed	Persistence
useWorkspaceStore	<code>runtimes/workspace/store.ts</code>	Workspaces[] (name, type, identity, status, layout, dirty, saveStatus, validationErrors), activeId, workspace lifecycle (open/close/suspend/resume/pin/activate), dirty tracking, hydration	localStorage
useLivingStore	<code>components/living-workspace/living-store.ts</code>	founderName, realityEvents[], observations[], recommendations[], executions[], livingObjects[], expandedObjectId, sidebarCollapsed, commandOpen, commandInput, lastUpdated, realityPollTime, activeExecutions, fetch/poll actions, founderPreferences, interactionHistory, reflectionMessages	In-memory + API polling

Only 2 Zustand stores exist. No duplication.

5. ALL REACT CONTEXTS/PROVIDERS

Context	File	State Provided	Used By
WorkspaceContext	<code>context/WorkspaceContext.tsx</code>	workspaces, activeWorkspace,	<code>app.tsx</code> (wraps HomePage),

		switchWorkspace, createWorkspace, loading, error, refresh	components via <code>useWorkspace()</code>
NotificationContext	<code>components/notifications/notification-context.tsx</code>	notifications, addNotification, dismissNotification, clearAll	Any component via <code>useNotifications()</code>
ShellContext	<code>components/workspace/workspace-shell.tsx</code>	contextPanelCollapsed, toggleContextPanel, workspaceContext	Workspace shell components via <code>useShellContext()</code>
TokenCtx	<code>tokens/token-provider.tsx</code>	Empty <code>{}</code> , CSS variable injection via <code><style></code>	<code>app.tsx</code> (wraps everything)

Duplication found: None directly, but WorkspaceContext (legacy) and useWorkspaceStore (canonical runtime) manage overlapping state.

6. ALL API CLIENT FILES & ENDPOINTS

File	Base Path	Endpoints	Auth
<code>api/client.ts</code>	<code>/api/v1</code>	<code>signin</code> , <code>query</code> , <code>forgotPassword</code> , <code>resetPassword</code> , <code>requestVerification</code> , <code>verifyEmail</code> , <code>getInvitation</code>	X-Identity-Id
<code>api/ai-chat.ts</code>	<code>/api/v1/ai</code>	<code>POST /chat</code>	None
<code>api/objects.ts</code>	<code>/api/v1/objects</code>	CRUD for all object types + upload	X-Identity-Id + X-Workspace-Id
<code>api/integrations.ts</code>	<code>/api/v1/integration</code>	Integration CRUD, social accounts, scheduled posts, ad campaigns, content gen	X-Identity-Id
<code>api/workspace.ts</code>	<code>/api/v1/objects</code>	Workspace CRUD	X-Identity-Id + X-Workspace-Id
<code>api/supabase.ts</code>	(Supabase)	<code>signInWithEmail</code> , <code>signUpWithEmail</code> , <code>signInWithGoogle</code> , <code>signInWithGithub</code> , <code>signOut</code> , <code>getSession</code>	Supabase JWT
<code>api/use-ai-presence.ts</code>	<code>/api/v1/ai/insights</code>	Polls insights	credentials: 'include'
<code>api/use-realtime-sync.ts</code>	<code>/api/v1/events</code>	Delta events polling	credentials: 'include'
<code>api/use-query.ts</code>	(generic)	Generic GET	credentials: 'include'
<code>api/fetch-with-auth.ts</code>	(generic)	Generic fetch wrapper	X-Identity-Id

Duplication: `client.ts` and `fetch-with-auth.ts` both implement the same X-Identity-Id pattern. `objects.ts` and `workspace.ts` both implement workspace CRUD with the same auth pattern.

7. ALL HOOKS

Hook	File	Purpose
<code>useQuery<T></code>	<code>api/use-query.ts</code>	Generic data fetching with loading/error state, event-bus refresh
<code>useMutation</code>	<code>api/use-query.ts</code>	Generic POST/PUT/DELETE with XHR progress tracking
<code>useAIPresence</code>	<code>api/use-ai-presence.ts</code>	Polls AI insights every 45s, deduplicates, emits on event bus
<code>useRealtimeSync</code>	<code>api/use-realtime-sync.ts</code>	Polls delta events every 15s, emits created/updated events
<code>useWorkspaceMemory</code>	<code>api/use-workspace-memory.ts</code>	Saves/restores workspace UI state to sessionStorage
<code>useRuntime</code>	<code>hooks/runtime-hooks.ts</code>	Get runtime by ID from orchestrator
<code>useRuntimes</code>	<code>hooks/runtime-hooks.ts</code>	Get all registered runtimes
<code>useRuntimesReady</code>	<code>hooks/runtime-hooks.ts</code>	Check if all runtimes ready
<code>useRuntimeHealth</code>	<code>hooks/runtime-hooks.ts</code>	Aggregate runtime health summary
<code>useOpenWorkspace</code>	<code>hooks/workspace-hooks.ts</code>	Get workspace store's <code>open</code> function
<code>useActiveWorkspace</code>	<code>hooks/workspace-hooks.ts</code>	Get currently active workspace
<code>useWorkspaceList</code>	<code>hooks/workspace-hooks.ts</code>	Get all workspaces
<code>useWorkspaceActions</code>	<code>hooks/workspace-hooks.ts</code>	Get all workspace action functions
<code>useWorkspaceHydration</code>	<code>hooks/workspace-hooks.ts</code>	Hydrate workspace store on mount
<code>useCommandPalette</code>	<code>hooks/workspace-hooks.ts</code>	Get command palette functions (<code>openObject</code> , <code>openHome</code> , <code>openSearch</code>)

<code>useWorkspaceDirtyState</code>	<code>hooks/workspace-hooks.ts</code>	Get dirty state tracking
<code>useActiveWorkspaceStatus</code>	<code>hooks/workspace-hooks.ts</code>	Get active workspace status/details
<code>useWorkspace</code>	<code>context/WorkspaceContext.tsx</code>	Get WorkspaceContext value (legacy)
<code>useNotifications</code>	<code>components/notifications/notification-context.tsx</code>	Get notification context
<code>useShellContext</code>	<code>components/workspace/workspace-shell.tsx</code>	Get shell context (collapsed state)
<code>useToken</code>	<code>tokens/token-provider.tsx</code>	Get token context (empty)
<code>useRuntimeState</code>	<code>runtimes/state-fabric.ts</code>	Subscribe to runtime state changes

Duplication: None at the hook level, but `useAIPresence` (polling) and `useRealtimeSync` (polling) are separate polling mechanisms that could be unified.

8. ALL POLLING / SSE / REAL-TIME MECHANISMS

Mechanism	File	Interval	Endpoint	Purpose
AI Presence Polling	<code>api/use-ai-presence.ts</code>	45s	<code>/api/v1/ai/insights</code>	Proactive AI insights
Realtime Delta Sync	<code>api/use-realtime-sync.ts</code>	15s	<code>/api/v1/events?since=...</code>	Delta object updates
Notification Unread Poll	<code>components/notifications/notification-bell.tsx</code>	30s	<code>/api/v1/integration/notifications/unread-count</code>	Unread notification count
WorkspaceBar Notification Poll	<code>components/workspace/workspace-bar.tsx</code>	On-open	<code>/api/notifications?limit=5</code>	Notification list
Runtime Health Poll	<code>hooks/runtime-hooks.ts</code>	5s	(orchestrator internal)	Runtime availability
Living Store Reality Poll	<code>components/living-workspace/living-store.ts</code>	15s	(Reality Engine API)	Continuous reality updates
Living Store Insight Poll	<code>components/living-workspace/living-store.ts</code>	30s	(AI insights API)	AI observation updates

Duplication: 7 separate polling mechanisms — no SSE, no WebSocket. Every component polls independently. The `use-realtime-sync.ts` (15s) and the `living-store.ts` reality poll (15s) may be duplicating the same delta sync work.

Analysis: This is a structural problem — the system has no single real-time transport layer. Every hook implements its own polling loop with its own interval, endpoint, and state management.

9. ALL NOTIFICATION / BELL / ALERT COMPONENTS

Component	File	Purpose
NotificationBell	<code>components/notifications/notification-bell.tsx</code>	Bell icon + unread badge, polls <code>/api/v1/integration/notifications/unread-count</code> every 30s
NotificationProvider	<code>components/notifications/notification-context.tsx</code>	Global notification context — <code>addNotification</code> , <code>dismissNotification</code> , <code>clearAll</code> , <code>auto-dismiss</code> , maintains last 50
NotificationHistory	<code>components/notifications/notification-history.tsx</code>	Mantine-based panel showing all past notifications with <code>clear-all</code> + individual <code>dismiss</code>
NotificationToast	<code>components/notifications/notification-toast.tsx</code>	Re-export of <code>notification-toast-impl</code>
NotificationToast (impl)	<code>components/notifications/notification-toast-impl.tsx</code>	Standalone toast overlay (inline styles)
Notifications (Mantine)	<code>app.tsx</code> (via <code>@mantine/notifications</code>)	Mantine's built-in Notifications component (<code>showNotification</code>)
WorkspaceBar Notification	<code>components/workspace/workspace-bar.tsx</code>	Inline notification dropdown with <code>fetch-unread</code> + <code>mark-read</code>

Duplication: MAJOR. There are THREE notification UX systems: 1. **Custom NotificationContext** (`notification-context.tsx`) — custom toast, history panel, inline styles 2. **Mantine Notifications** (`@mantine/notifications`) — used in `homepage.tsx` via `showNotification` 3. **Inline WorkspaceBar Notifications** (`workspace-bar.tsx`) — its own fetch + dropdown

Verdict: NotificationContext is the canonical custom system. WorkspaceBar should use it instead of inline notification management. Mantine's system should be the fallback.

10. ALL ROUTING / NAVIGATION MECHANISMS

Mechanism	File	How It Works
Path-based routing	app.tsx	<code>window.location.pathname</code> <code>/auth/</code> → <code>AuthRouter</code> , <code>/living</code> → <code>LivingWorkspace</code> , <code>/</code> → <code>HomePage</code>
Event dispatch routing	components/ai/command-palette.tsx	<code>window.dispatchEvent(new CustomEvent('shunya:action', {detail}))</code>
Workspace store navigation	runtimes/workspace/store.ts	<code>open()</code> creates/activates workspaces, emits <code>WorkspaceOpened</code> event
Module registry search	runtimes/module-registry.ts	<code>searchAll()</code> queries all modules for search results
Composition Engine	runtimes/composition/engine.ts	Renders workspace content based on type
Window location redirects	components/executive-home/command-surface.tsx	<code>window.location.href = route</code> for <code>/open</code> commands
Hyperlink-based	components/public/homepage.tsx	Mantine Card <code>onClick</code> → space selection via state

Duplication: Mixed. No react-router. Path-based, event-based, and store-based navigation all coexist. The `command-surface.tsx` does `window.location.href = route` which is a hard navigation — incompatible with SPA.

11. DUPLICATION SCORECARD — SUMMARY

MAJOR DUPLICATIONS (must resolve):

#	Duplicates	Files	Canonical	Action
1	Three Event Buses	api/event-bus.ts , lib/event-bus.ts , runtimes/event-bus.ts	runtimes/event-bus.ts	Remove <code>lib/event-bus.ts</code> , merge <code>api/</code> onto <code>runtimes/</code>
2	Four Command Surfaces	executive-home/command-surface.tsx , living-workspace/command-surface.tsx , ai/command-bar.tsx , ai/command-palette.tsx	living-workspace/command-surface.tsx	Remove <code>ai/command-bar.tsx</code> and <code>executive-home/command-surface.tsx</code>
3	Three Notification Systems	notification-context.tsx + @mantine/notifications + workspace-bar.tsx inline	notification-context.tsx	Remove inline notification from <code>WorkspaceBar</code>
4	Two AI Copilot panels	copilot/ai-copilot.tsx , living-workspace/ai-presence-panel.tsx	ai-presence-panel.tsx	Remove <code>copilot/ai-copilot.tsx</code>
5	Three Session/Auth systems	api/session.ts , api/supabase-session.ts , api/supabase.ts	api/supabase.ts	Remove <code>api/session.ts</code> and <code>api/supabase-session.ts</code>
6	Two Auth-fetch wrappers	api/client.ts , api/fetch-with-auth.ts	api/client.ts	Remove <code>api/fetch-with-auth.ts</code>
7	Two Component Registries	components/registry.ts , lib/component-registry.ts	lib/component-registry.ts	Remove <code>components/registry.ts</code>
8	Two Workspace Bar implementations	public/os-bar.tsx , workspace/workspace-bar.tsx	workspace/workspace-bar.tsx	Remove <code>public/os-bar.tsx</code>

MODERATE ISSUES:

#	Issue	Details
9	Polling overload	7 independent polling loops with different intervals (5s/15s/30s/45s)
10	CSS file duplication	5 CSS files across components: <code>living-styles.css</code> , <code>living-os.css</code> , <code>unified-os.css</code> , <code>auth-styles.ts</code> , <code>onboarding-styles.ts</code>
11	Error Boundary duplication	<code>main.tsx</code> has a React class <code>ErrorBoundary</code> AND <code>components/error-boundary.tsx</code> AND <code>components/ui/error-fallback.tsx</code>
12	WorkspaceContext vs runtime store	<code>context/WorkspaceContext.tsx</code> and <code>runtimes/workspace/store.ts</code> manage overlapping workspace state

FILES DEAD / UNREFERENCED:

File	Reason
lib/event-bus.ts	No imports found
lib/component-registry.ts	No imports found (components/registry.ts used instead)
api/profile.ts	Used by profile-switcher but duplicates session.ts patterns
components/error-boundary.tsx	main.tsx has its own ErrorBoundary
hooks/runtime-hooks.ts	Has its own polling loop when runtimes/state-fabric has useRuntimeState
components/executive/executive/index.tsx	Separate from executive/index.tsx

12. ARCHITECTURAL RECOMMENDATIONS

Immediate (LX-06 blockers):

1. **Consolidate to one Event Bus:** `runtimes/event-bus.ts` is canonical. Migrate `useAIPresence` , `useQuery` , `useRealtimeSync` to use it.
2. **Remove duplicate AI copilot:** Delete `components/copilot/ai-copilot.tsx` .
3. **Remove duplicate command bar:** Delete `components/ai/command-bar.tsx` .
4. **Remove duplicate auth-fetch:** Delete `api/fetch-with-auth.ts` .
5. **Remove duplicate session:** Delete `api/session.ts` and `api/supabase-session.ts` .

Short-term:

1. **Unify notification system:** Make NotificationContext canonical. Remove inline notification from WorkspaceBar.
2. **Unify component registries:** Move `components/registry.ts` content into `lib/component-registry.ts` .
3. **Unify error boundaries:** Use only one ErrorBoundary pattern.
4. **Reduce polling chaos:** Build a single `RealityEngine` transport hook that SSE-polls and distributes to all subscribers.
5. **Remove legacy OS bar:** Once WorkspaceBar is stable, remove `public/os-bar.tsx` .

Files to remove (total savings: ~12 files):

```
lib/event-bus.ts
components/registry.ts
components/copilot/ai-copilot.tsx
components/ai/command-bar.tsx
components/executive-home/command-surface.tsx (after migration)
components/executive-home/executive-home.tsx (after migration)
api/fetch-with-auth.ts
api/session.ts
api/supabase-session.ts
components/public/os-bar.tsx (after migration)
components/error-boundary.tsx
api/event-bus.ts (after migration)
```